



PEC 1 - Representación de problemas

Presentación

Primera PEC del curso de Inteligencia Artificial.

Competencias

En esta PEC se trabajaran les siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.

Objetivos

Esta PEC pretende evaluar vuestros conocimientos sobre formalización de problemas y búsqueda básica sobre espacio de estados.

Descripción de la PEC

Mirad el siguiente vídeo: <https://youtu.be/aWlQYlIBZDs>

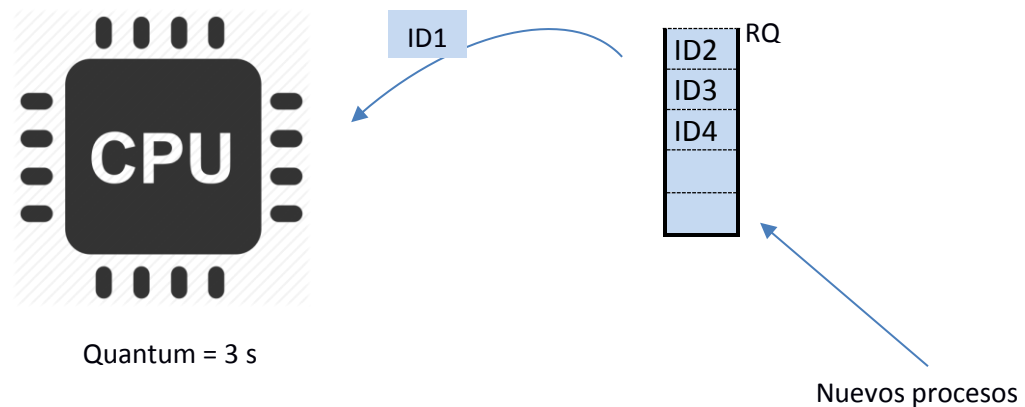
Se trata de la explicación del conocido algoritmo de planificación de CPU *Round Robin*. En esta PEC nos vamos a basar en alguna variante de esta técnica.

Consideremos, como variante inicial, una muy simple como la que se muestra en la siguiente figura. Como podemos ver, simplemente disponemos de una cola donde los procesos esperan a ser despachados a la CPU. La llamaremos *Request Queue (RQ)*. Por otro lado, contamos con la CPU, la unidad encargada de ejecutar los procesos.

En esta primera aproximación, a diferencia de lo mostrado en el vídeo, nuestro sistema no registra ni el tiempo de llegada de los procesos a la cola, ni tiene información sobre cuántos ciclos de CPU (cuánto tiempo) necesitan para finalizar, sino que la cola se va a gestionar mediante una simple estrategia FIFO



(*First In, First Out*). De este modo, los procesos se van despachando a la CPU a medida que llegan.



Una vez un proceso tiene asignada la CPU, se ejecutará durante un *quantum* de tiempo. Una vez consumido el *quantum*, volverá a la cola, en el último lugar y automáticamente empezará el siguiente proceso. Podemos considerar que este cambio se realiza instantáneamente y no consume tiempo. El *quantum* de nuestro sistema es de 3 s.

Como nuestro sistema es bastante limitado, el Sistema Operativo tan sólo permite una cola de 5 procesos en espera, y la CPU tan solo puede estar sin ejecutar ningún proceso si no hay ningún proceso en la RQ. Si un proceso aparece y la RQ está vacía, éste pasa directamente a ser ejecutado. Si un proceso consume su *quantum*, aún no ha terminado, y la RQ está vacía, se le renueva el *quantum* a 3 s de nuevo.

Eso sí, los procesos necesitan ser identificados, por lo que, al llegar al sistema, recibirán un identificador. Este identificador será reutilizado por otros procesos cuando el proceso al que identificaba inicialmente abandone el sistema (complete su ejecución). Nunca habrá identificadores repetidos entre la RQ y la CPU.

En este contexto, debéis responder a las siguientes cuestiones:

1. (2 puntos, 1 por apartado)

- a) Definid una representación del sistema, de manera que en cualquier instante se pueda determinar su estado. Es decir, en cualquier instante debe quedar claro el estado de la cola, del procesador, y el *quantum* restante del proceso en ejecución.
- b) Mostrad la representación completa, según vuestra respuesta, de un estado cualquiera en el que la cola no esté llena y el *quantum* restante sea 1 segundo.

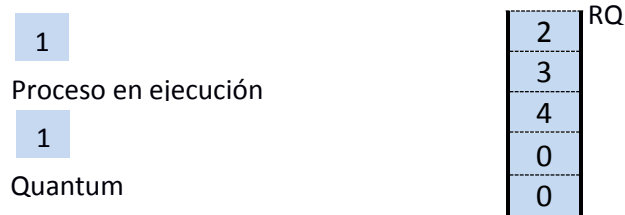


a) Podemos formalizar el problema con una lista (matriz unidimensional) de 5 celdas que representaría la RQ, más una celda aparte para el proceso en ejecución (que podría formar parte sin problemas de esta misma lista, siempre y cuando controlemos la posición), y otra celda/variable que contenga el *quantum* restante.

Los elementos de la celda podrán tomar 7 valores distintos; los correspondientes a los 6 diferentes identificadores (por simplicidad, tomaremos los valores del 1 al 6), más uno que indique que la celda está vacía (tomaremos el valor 0). La variable destinada al *quantum* tendrá tres valores distintos: 1, 2 y 3.

Se podría considerar una representación más compleja que respete la estructura FIFO de la cola (imposibilidad de que haya huecos vacíos en posiciones “intermedias” de la cola, entre otros aspectos), pero en esta solución traspasamos esta complejidad a los operadores, como veremos a continuación.

b) Representación:



2. (2 puntos, 1 por apartado)

- ¿Cuántos estados posibles tiene vuestra representación? ¿Todos los estados posibles son válidos? ¿Todos los estados son accesibles desde cualquier otro estado?
- Definid los operadores necesarios que cambian los estados del sistema de acuerdo a lo descrito (pensad en qué variables de vuestra representación necesitan ser alteradas durante el funcionamiento del sistema). ¿Son parametrizados, estos operadores?

a) Teniendo en cuenta lo respondido en 1a), el número de estados posibles en esta representación es de $7^5 \times 7 \times 3 = 352947$ estados posibles. No todos los estados posibles son válidos; no son válidos aquellos estados con 0 en proceso en ejecución y una cola diferente a todo ceros, ni tampoco lo es ningún estado



en el que haya algún identificador de RQ repetido o que también esté como identificador del proceso en ejecución.

Obviamente, no podemos acceder de cualquier estado a cualquier otro estado. Por ejemplo, no podemos acceder de un estado donde el *quantum* restante es 1 a uno en el que el *quantum* restante es 2.

b) Un operador es un procedimiento o función que transforma un estado válido en otro válido. En nuestro caso, para que el sistema funcione, podríamos definir 3 operadores:

- Operador decremento de *quantum*: considerando Q_r como el valor del *quantum* restante, es un operador que cada vez que transcurre un segundo, si hay un programa en ejecución, decrementa el valor Q_r , y si Q_r es 1, lo transforma en 3. No necesita parametrización (es un simple decremento de una de las variables de estado, conocida de antemano).
- Operador de admisión: si aparece un nuevo proceso y RQ no está llena ni está vacía, se insertará en ella, en la primera posición libre. Si está vacía y la CPU también, pasará directamente a CPU. En ambos casos se le asignará el primer identificador que no esté siendo usado dentro del conjunto $\{1..6\}$. Requerirá como parámetro el nuevo identificador que se le asigne.
- Operador de cambio de proceso: si el proceso en ejecución termina antes de consumir el *quantum*, en el siguiente instante de tiempo en CPU estará ubicado el proceso de RQ que corresponda, y Q_r actualizado a 3. Si al proceso en ejecución se le termina el *quantum*, pasará a la primera posición libre de RQ y pasará a ejecutarse el proceso que corresponda de RQ. Requerirá, como parámetros, los identificadores de los procesos que se intercambian.

3. (2 puntos) El sistema ha sido mejorado, de manera que ahora los procesos, pueden indicar dependencias. Es decir, un proceso B puede indicar que para iniciar su ejecución necesita primero que el proceso A haya terminado, por ejemplo. Así pues, ahora disponemos de una cola adicional (también de tamaño 5), donde se ubicarán los procesos que están a la espera de que termine alguno de los procesos de RQ. Los procesos con dependencias aguardarán en esta nueva cola (que podemos denominar DQ – *dependence queue*) hasta que el proceso del que dependen termine su ejecución. Una vez éste termine, serán traspasados a RQ al instante (si hay espacio).

Como hasta ahora, los procesos en RQ irán alternando su ejecución en intervalos marcados por el *quantum*. Sin embargo, la gestión de RQ ahora no tiene porqué seguir una política FIFO, sino que cualquiera de los procesos existentes puede ser seleccionado a conveniencia para consumir el siguiente intervalo de *quantum*, incluso un proceso proveniente de DQ puede ejecutarse



inmediatamente en CPU si en RQ había espacio o el proceso que anteriormente estaba en CPU ha terminado su ejecución.

Por simplicidad, supondremos que un proceso puede tener dependencia de tan solo otro proceso (no existirá un proceso X que requiera que antes hayan terminado un proceso Y y otro Z).

Adaptad vuestra representación anterior, explicando los cambios que se deben introducir para reflejar el nuevo sistema. ¿Siguen siendo útiles los operadores anteriormente definidos? Si no es el caso, detallad cómo se ven modificados, o si es necesario añadir alguno nuevo.

Las diferencias fundamentales son que hemos de añadir una matriz de 5x2 para representar la nueva cola DQ. Asimismo, el número de identificadores de proceso aumenta de 6 a 11.

Respecto a los operadores empleados en esta solución (obviamente, esto variará en función de las respuestas dadas en los ejercicios 1 y 2), habría que o bien añadir un nuevo operador que consulte en cada instante si las condiciones para que un proceso de DQ pase a RQ (y en caso positivo, realizarlo), o bien ampliar la funcionalidad del operador de “cambio de proceso” anteriormente propuesto, para que además realice esta acción.

4. (2 puntos) El objetivo principal del Sistema Operativo con esta nueva configuración es que la DQ disponga siempre del máximo número posible de *slots* libres, y que sea vaciada lo antes posible, puesto que esta cola es ahora lo que limita la aceptación de nuevos procesos enviados por los usuarios. Con esta única premisa, el Sistema Operativo debe ir enviando a ejecutar en CPU los procesos en RQ.

Partiendo de esta base, nos interesa diseñar un sistema de decisión basado en la exploración de los estados sucesivos posibles (dado un estado inicial), para decidir en cada intervalo de *quantum* qué proceso de RQ es enviado a CPU.

Responded justificadamente:

- a) ¿Se trata de un problema de satisfacción de restricciones, o de planificación?

Se trata de un problema de planificación, puesto que lo que esperamos como solución es una secuencia de acciones que nos lleven desde un estado inicial a uno objetivo.

- b) ¿Es posible que, en la exploración de estados, entremos en una situación de bucle?

Esta pregunta está orientada a evaluar la correcta reflexión del alumno sobre el problema planteado y su solución propuesta. Existen multitud de respuestas correctas.



Formalmente, y si nos ceñimos estrictamente a la definición del problema, se puede entrar en bucles al recorrer el espacio de estados, dado que carecemos de información acerca del tiempo restante de ejecución que le queda a cada proceso. En la práctica, no sería posible entrar en bucle, puesto que el hecho que la CPU nunca puede estar desocupada, a no ser que no haya ningún proceso en ninguna cola, hace que los procesos vayan completando su ejecución irremediabilmente, por lo que ningún estado nunca sería igual que otro.

También conviene destacar que, de nuevo, formalmente sí que se podrían repetir estados, aunque a nuestra representación se le añadiese información acerca del tiempo restante, puesto que los identificadores son reutilizables. En este caso, no obstante, sería fácil determinar si se trata realmente de una repetición de estado, ya que la aparición de nuevos procesos la deberíamos saber de antemano al plantear el problema. Es decir, debido a la naturaleza estocástica de aparición de nuevos procesos, nuestro planificador tan solo podrá hacer una búsqueda entre los estados posibles que derivan de un determinado estado inicial, sin contemplar la aparición de nuevos procesos dentro del árbol de búsqueda.

- c) ¿Qué estrategias debemos implementar para evitarlo?

Debido a lo explicado anteriormente, no deberíamos evitarlo.

En el caso general de otros problemas de representación y búsqueda de soluciones en el espacio de estados, deberíamos tener en cuenta cuándo la ejecución de un operador puede dar como resultado un estado antecesor al actual.

- d) ¿Qué es más conveniente en este caso: realizar una búsqueda en anchura o en profundidad?

Puesto que lo que buscamos es tener DQ vacía lo antes posible, y no se nos indica que tengamos limitación de memoria, nos interesa que nuestra búsqueda sea en anchura.

5. (2 puntos) Imaginemos un estado en el que un proceso ID1 acaba de empezar su ejecución, los procesos ID2, ID3 e ID4 están en RQ, y en DQ tenemos al proceso ID5 con dependencia de ID3, los procesos ID6 e ID7 ambos con dependencia de ID4, el proceso ID8 con dependencia de ID7, e ID9 con dependencia de ID3. Consideraremos que este estado existe para el instante de tiempo $t=0$. Todos los procesos terminan su ejecución en 3 s, que es el *quantum* establecido.

De acuerdo con la premisa del ejercicio 4, y con vuestra representación del ejercicio 3, mostrad los estados para todos los instantes en que el *quantum* restante es 3 s (es decir, para $t=0$, $t=3$, $t=6...$) hasta que no quede ningún proceso en DQ.



t=0

1	Proceso en ejecución	ID	RQ	ID	Dep.	DQ
		2		5	3	
		3		6	4	
		4		7	4	
3	Quantum	0		8	7	
		0		9	3	

t=3

4	Proceso en ejecución	ID	RQ	ID	Dep.	DQ
		2		5	3	
		3		6	4	
3	Quantum	0		7	4	
		0		8	7	
		0		9	3	

t=6

3	Proceso en ejecución	ID	RQ	ID	Dep.	DQ
		2		5	3	
		6		8	7	
3	Quantum	7		9	3	
		0		0	0	
		0		0	0	



t=9

7		ID		ID	Dep.	
		2	RQ	8	7	DQ
		6		0	0	
		5		0	0	
		9		0	0	
		0		0	0	

Proceso en ejecución

3

Quantum

t=12

2		ID		ID	Dep.	
		6	RQ	0	0	DQ
		5		0	0	
		9		0	0	
		8		0	0	
		0		0	0	

Proceso en ejecución

3

Quantum

Nota: en t=3 tanto podría haberse ejecutado el proceso 4 como el 3 y acabaríamos en el mismo instante; se escoge 4 porque un proceso que depende de él es a su vez dependencia de otro.

Recursos

Módulo 1 y Módulo 2, temas 1-3, de los materiales de la asignatura.

Criterios de valoración

Cada uno de los ejercicios tiene una valoración de 2 puntos.

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.



Formato y fecha de entrega

Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.

Se ha de entregar la solución en un fichero PDF una de las plantillas entregadas conjuntamente con este enunciado. Adjuntad el fichero a un mensaje en el apartado Entrega y Registro de EC (REC). El nombre del fichero ha de ser ApellidosNombre_IA_PEC1 con la extensión .pdf (PDF).

La fecha límite de entrega es el **23 de marzo (inclusive)**.

Nota: **Propiedad intelectual**

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.